

Relatório de Auditoria de Segurança — Synergia

Isolamento de tenant, autorização no servidor, IDOR, segredos expostos e XSS em uma aplicação Kotlin/Spring Boot + Angular.

Data	18/09/2026
Repositório	github.com/Gharantes/repo-tests (público)
Versão auditada	1e166f9 (branch teste/protecao)
Resultado	24 achados: 3 crítica, 9 alta, 4 média, 5 baixa, 3 informativa; 11 pontos fortes

Escopo auditado

- Backend Kotlin/Spring Boot: os 12 controllers em backend/src/main/kotlin/br/com/synergia/rest (35 handlers, todos lidos), serviços, DTOs, entidades JPA, os 17 arquivos .sql e os perfis application-*.yaml.
- Frontend Angular 19/Nx: rotas, guard, SessionService, todos os templates .html (busca por sinks de HTML/URL) e o bundle já compilado em frontend/dist.
- Deploy e automação: backend/Dockerfile, .github/workflows/ci.yml, backend/build.gradle.kts, frontend/nx.json, configurações de IDE versionadas (.idea, db/.idea).
- Documentação (docs/, college/, READMEs) e todo o histórico git (203 commits, todas as branches) atrás de segredos commitados.

Stack detectada

Camada	Detectado
Linguagem / framework	Kotlin 1.9 + Spring Boot 3.5 (spring-boot-starter-web), JDK 21
Acesso a dados	Spring Data JPA (repositórios) + NamedParameterJdbcTemplate com SQL em arquivos .sql; PostgreSQL
Autenticação	Nenhuma no servidor: sem Spring Security, sem sessão, sem token. O login devolve só IDs, que o frontend guarda no localStorage e reenvia como parâmetros
Isolamento de tenant	Filtro manual por id_tenant nas queries, com o valor vindo do próprio cliente (query string ou corpo). Não há RLS nem middleware de tenant
Frontend	Angular 19 standalone (Nx), cliente gerado por OpenAPI Generator; sem lib de sanitização extra
Deploy / CI	Dockerfile multi-stage (Render, perfil datasource-render), GitHub Actions com Postgres efêmero; sem Helm/Terraform/docker-compose

Nota metodológica: como cada categoria foi mapeada para a stack

Categoria	Equivalente nesta stack e como foi verificado
1. Banco sem tranca	Sem Supabase/RLS. O mecanismo de isolamento é o filtro WHERE id_tenant = :id_tenant. Foi verificado de onde vem esse valor (servidor ou cliente) em cada listagem, e se as queries por relacionamento também filtram por tenant.

Categoria	Equivalente nesta stack e como foi verificado
2. Permissão no navegador	Mapeados todos os gates do Angular (guard HasActiveTenant, menu Administração, SessionService) e cruzados com os endpoints correspondentes. Procurado qualquer uso do enum AuthPermissionsEnum no backend.
3. IDOR	Percorridos os 35 handlers dos 12 controllers. Para cada ID recebido (path, query ou body) foi verificado se há checagem de posse/tenant antes do findById/save/deleteById.
4. Chaves expostas	Busca por padrões de segredo em todos os arquivos versionados, nos perfis Spring (incluindo defaults \${VAR:valor}), no CI, nos scripts, na documentação, em git log -p de todas as branches e no bundle compilado em frontend/dist.
5. XSS	Busca por innerHTML, bypassSecurityTrust*, DomSanitizer, eval/new Function, [href]/[src]/[style] com dado do usuário e markdown. No backend: respostas HTML, e-mails e templates (não existem; a API só devolve JSON).

Só entram no relatório achados verificados no código desta versão. Severidade considera o deploy público em repo-tests.onrender.com e o repositório público no GitHub. Caminhos longos aparecem abreviados nas tabelas (backend/.../synergia/ = backend/src/main/kotlin/br/com/synergia/; backend/.../resources/ = backend/src/main/resources/); as issues trazem o caminho completo.

Resumo executivo

O Synergia não tem autenticação no servidor. O login só confere a senha e devolve IDs; a partir daí o navegador envia o **id-tenant** e o **idAccount** que quiser, e o backend confia. Por isso o filtro por tenant, que existe em todas as listagens, não isola nada, e nenhuma rota de escrita confere posse. Somado a um modo de login que dispensa a senha e a um update de conta sem checagem, qualquer pessoa na internet consegue ler, alterar e apagar dados de todas as instituições e assumir a conta ADMIN de cada uma. Do lado positivo, não há SQL injection nem XSS: as queries são parametrizadas e o frontend usa apenas interpolação do Angular. O perfil de produção também não carrega segredos.

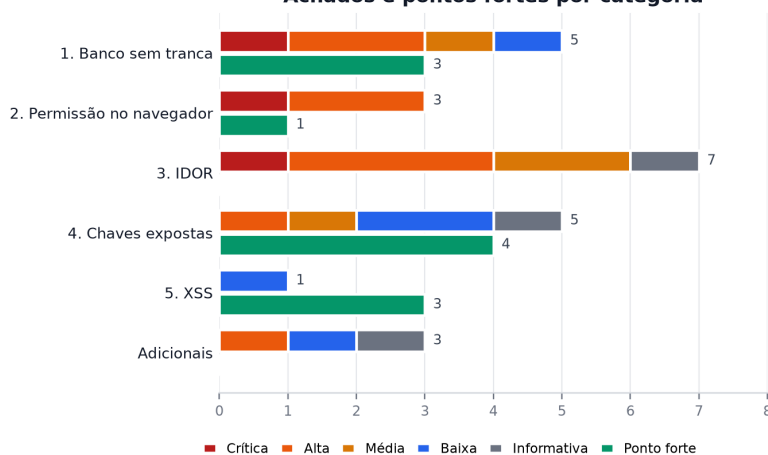
3	9	4	5	3	11
Crítica	Alta	Média	Baixa	Informativa	Pontos fortes

Achados por severidade



■ Crítica (3) ■ Alta (9) ■ Média (4) ■ Baixa (5) ■ Informativa (3)

Achados e pontos fortes por categoria



■ Crítica ■ Alta ■ Média ■ Baixa ■ Informativa ■ Ponto forte

Categoria	Crítica	Alta	Média	Baixa	Informativa	Total	Fortes
1. Banco sem tranca	1	2	1	1	–	5	3
2. Permissão no navegador	1	2	–	–	–	3	1
3. IDOR	1	3	2	–	1	7	–
4. Chaves expostas	–	1	1	2	1	5	4
5. XSS	–	–	–	1	–	1	3
Adicionais (fora das 5)	–	1	–	1	1	3	–

Pontos fortes

O que foi verificado e está correto. Cada item é evidência de cobertura da auditoria.

	Categoria	Controle e evidência
PONTO FORTE	Banco sem tranca	SQL sempre parametrizado. Todas as queries .sql que recebem entrada usam parâmetros nomeados do NamedParameterJdbcTemplate (ex.: list-accounts.sql:9-10). O único replace de texto (ActionAttributePermissionsSqlService.kt:15-16) concatena List<Long> já tipada pelo Jackson; não há SQL injection.
PONTO FORTE	Banco sem tranca	Login restrito ao tenant. check-login-information.sql:11 filtra a.id_tenant e a resposta (LoginInformationResponseDto) não inclui a senha; AccountDto também não, o que os testes de integração conferem.
PONTO FORTE	Banco sem tranca	Filtros de tenant presentes nas listagens. list-*.by-tenant.sql e list-tags.sql já têm WHERE id_tenant; a correção é trocar a origem do valor (sessão em vez de parâmetro), não reescrever as queries.
PONTO FORTE	Chaves expostas	Perfil de produção sem segredos. application-datasource-render.yaml:3-5,12 usa \${DATABASE_URL}, \${DATABASE_USERNAME}, \${DATABASE_PASSWORD} e \${FRONTEND_URL} sem valor padrão: sem a variável, a aplicação não sobe.
PONTO FORTE	Chaves expostas	Bundle do frontend limpo. Varredura de frontend/dist e dos environment*.ts: só a URL pública da API (environment.prod.ts:2). Nenhuma chave, token ou JWT embutido.
PONTO FORTE	Chaves expostas	CI com privilégio mínimo. ci.yml:25-26 fixa permissions: contents: read; não há secrets.* no workflow; o Dockerfile não copia arquivos de configuração locais além de src/.
PONTO FORTE	Chaves expostas	CI sem injeção de script. ci.yml:63-68 passa o título do PR por env: e usa grep -qF em vez de interpolar \${ { github.event.pull_request.title } } dentro do shell.
PONTO FORTE	XSS	Templates Angular só com interpolação. Nenhum innerHTML, outerHTML, bypassSecurityTrust*, DomSanitizer, eval ou new Function em frontend/src. Títulos e descrições entram por {{ }}, escapados pelo Angular.
PONTO FORTE	XSS	URLs e estilos passam pelo sanitizador do Angular. safe-image.component.html:4 usa [src], sanitizado; [style] na linha 11 recebe bannerColor gerado no servidor (ColorsEnum.randomHex), não texto do usuário.
PONTO FORTE	XSS	Backend não gera HTML. A API só responde JSON; não há envio de e-mail, Thymeleaf ou templates. O header x-error remove CR/LF (ResponseMessenger.kt:27), o que evita injeção de cabeçalho.
PONTO FORTE	Permissão no navegador	CORS com origem única. CorsConfig.kt:14-17 libera uma origem configurável, sem curinga e sem allowCredentials. Não substitui autenticação, mas não abre a API para qualquer site via navegador.

Pontos fracos: os riscos centrais

	Risco	Por que importa
CRÍTICA	A identidade é do cliente, não do servidor	Sem Spring Security, o backend usa o id-tenant e o idAccount enviados pela requisição. Tudo o que depende de "quem é o usuário" pode ser falsificado com curl (C-01, A-03, A-04).
CRÍTICA	Tomada de conta em duas chamadas	O login aceita checkLastSeen=true e não compara a senha (C-02); o update de conta troca a senha de qualquer ID (C-03). Os logins e IDs necessários são públicos (A-01, M-01).
ALTA	Nenhuma rota de escrita confere posse	Update e delete de contas, tags, eventos, projetos e tenants operam sobre qualquer ID (A-05, A-06, A-07). As permissões do AuthPermissionsEnum existem mas nunca são aplicadas.

Risco		Por que importa
ALTA	Segredos no repositório público	Token read-write do Nx Cloud desde o primeiro commit (A-08) e a senha MaybeLater repetida em perfis, fallbacks, build e documentação (M-04), além de senhas no histórico (B-03).
ALTA	Senhas em texto puro	Qualquer leitura do banco expõe as senhas reais dos usuários (A-X1).

Cadeia de ataque de ponta a ponta: POST /api/entity-tenant/list-all-tenants → POST /api/entity-account/list-accounts-by-tenant?id-tenant=N (IDs e logins) → POST /api/entity-account/update/{id do ADMIN} com password nova → login normal como ADMIN. Nenhum passo exige credencial.

Achados detalhados por categoria

Ordenados por severidade dentro de cada categoria. A coluna de descrição traz o trecho de código e a condição de explorabilidade.

1. Banco sem tranca

Severidade	Arquivo:linha	Descrição
CRÍTICA	backend/build.gradle.kts:39-52 backend/.../synergia/SynergiaApplication.kt:8-14 backend/.../synergia/config/CorsConfig.kt:13-18 backend/.../synergia/rest/*.kt (todos)	C-01 · Backend sem autenticação: os 35 endpoints aceitam chamadas anônimas Não há Spring Security, filtro, interceptor nem token. O servidor não sabe quem está chamando, então o filtro por tenant usa o id-tenant que o próprio chamador envia. O CORS restringe só navegadores; curl, scripts e o Swagger ignoram CORS. É a causa raiz de quase todos os demais achados. <pre>dependencies { ... "spring-boot-starter-web" ... } // sem spring-boot-starter-security @RequestParam("id-tenant") idTenant: Long // tenant vem do cliente</pre> Explorabilidade: Qualquer pessoa na internet alcança https://repo-tests.onrender.com/api/... sem credencial.
ALTA	backend/.../synergia/rest/EntityAccountResource.kt:16-26 backend/.../resources/sql/page-list-accounts/list-accounts.sql:9	A-01 · Listagem de contas de qualquer tenant, com login e e-mail O id-tenant é escolhido pelo chamador. Iterando 1..N obtém-se login, nome e e-mail de todos os usuários de todas as instituições. Os logins alimentam diretamente o bypass de senha (C-02). <pre>@PostMapping("/list-accounts-by-tenant") fun listAccountsByTenant(@RequestParam("id-tenant") idTenant: Long, ...) WHERE a.id_tenant = :id_tenant</pre> Explorabilidade: Nenhuma condição: basta um POST sem corpo.
ALTA	backend/.../synergia/rest/EntityEventResource.kt:15-24 backend/.../synergia/rest/EntityProjectResource.kt:15-24 backend/.../synergia/rest/EntityTagResource.kt:21-38 backend/.../resources/sql/entity-event/list-events-by-tenant.sql:10 backend/.../resources/sql/entity-project/list-projects-by-tenant.sql:10 backend/.../resources/sql/page-list-tags/list-tags.sql:11	A-02 · Eventos, projetos e tags listados por id-tenant arbitrário Mesmo padrão de A-01: o filtro existe, mas o valor é controlado pelo cliente, então não isola nada. Conteúdo de instituições privadas (is_private = true) fica legível. <pre>@RequestParam("id-tenant") idTenant: Long WHERE e.id_tenant = :id_tenant / p.id_tenant = :id_tenant / id_tenant = :id_tenant</pre> Explorabilidade: Nenhuma condição.
MÉDIA	backend/.../synergia/rest/EntityTenantResource.kt:15-22 backend/.../synergia/libs/entityTenant/services/EntityTenantSqlService.kt:19-21	M-01 · list-all-tenants devolve todas as instituições, inclusive as privadas A flag is_private não é considerada. O endpoint entrega o mapa de IDs de tenant que alimenta A-01/A-02 e C-02 (o login precisa do idTenant). <pre>fun listAllTenants(text: String?): List<TenantDto> { return tenantRepository.findAll().map { it.toDto() } // ignora text e isPrivate</pre> Explorabilidade: Nenhuma condição; o próprio CI chama esse endpoint sem credencial (ci.yml:264).
BAIXA	backend/.../synergia/rest/EntityAccountResource.kt:36-46 backend/.../synergia/libs/entityAccount/services/EntityAccountService.kt:20-30	B-01 · Oráculo de existência de login/e-mail por tenant Responde true/false para qualquer login ou e-mail em qualquer tenant. Permite confirmar se um e-mail é cliente da plataforma. Impacto menor porque A-01 já vaza a lista inteira. <pre>@GetMapping("get-account-by-login-or-email") -> Boolean</pre> Explorabilidade: Nenhuma condição.

2. Permissão no navegador

Severidade	Arquivo:linha	Descrição
CRÍTICA	backend/.../resources/sql/page-login/check-login-information.sql:10-18 backend/.../synergia/libs/pageLogin/models/LoginInformationInputDto.kt:7 frontend/src/app/modules/page-login/route-login.component.ts:53-58 backend/test/.../synergia/integration/PageLoginIntegrationTest.kt:121-133	C-02 · Login sem senha: o cliente liga checkLastSeen e o SQL deixa de comparar a senha A decisão de exigir ou não a senha é tomada por um booleano enviado pelo navegador. Com idTenant e login (ambos públicos via M-01 e A-01), qualquer um entra como qualquer usuário que tenha logado nas últimas 12 horas, inclusive o ADMIN. O teste de integração consagra o comportamento ("sessão recente é aceita ... mesmo sem senha"). <pre>CASE WHEN :check_last_seen THEN (a.last_seen IS NOT NULL AND ((now() - a.last_seen) < interval '12 hours')) ELSE a.password = :password END</pre> Explorabilidade: Vítima com last_seen < 12h. Cada login bem-sucedido do atacante renova o last_seen, então a janela se estende indefinidamente.
ALTA	frontend/src/app/security/routing/has-active-tenant.ts:14-22 frontend/src/libs/services/src/lib/session.service.ts:36-53 frontend/src/app/layout/component-layout-sidebar/layout-sidebar.component.html:60-75 backend/.../synergia/libs/utils/commons/enums/AuthPermissionsEnum.kt:8-20	A-03 · Controle de acesso existe só no navegador (guard, localStorage e menu Administração) O único gate é o guard HasActiveTenant, que olha um signal preenchido a partir do localStorage. Não existe papel (isAdmin/canEdit) na UI: o menu Administração (Usuários, Tags, Permissões) aparece para todo usuário, e o backend não confere papel em nenhuma rota. O enum AuthPermissionsEnum (CREATE_USER, EDIT_EVENT_AS_NON_MEMBER, ...) não é referenciado em nenhum lugar do backend. <pre>if (this.sessionService.getTenantId()) { return true; } localStorage.setItem('login-data', loginData); {tenant:{id}, user:{id}}</pre> Explorabilidade: Qualquer usuário logado executa ações administrativas; editar login-data no localStorage troca de identidade.
ALTA	backend/.../synergia/libs/entityEvent/models/UpsertEventDto.kt:4-5 backend/.../synergia/libs/entityProject/models/UpsertProjectDto.kt:4-5 backend/.../synergia/libs/entityEvent/services/EntityEventService.kt:30-33 backend/.../synergia/libs/entityProject/services/EntityProjectService.kt:20-24 backend/.../synergia/libs/entityAccount/services/EntityAccountSqlService.kt:67-75 backend/.../synergia/libs/entityTag/services/EntityTagSqlService.kt:51-60 frontend/src/app/modules/page-upsert-event/connector/connector-upsert-event.ts:17 frontend/src/app/modules/page-upsert-project/connector/connector-upsert-project.ts:15	A-04 · Autor e tenant de novos registros definidos pelo corpo da requisição O frontend preenche idAccount com o usuário da sessão local e o backend confia. Um chamador cria contas, tags, eventos e projetos dentro de outra instituição e nomeia qualquer conta como Organizador/Líder. As tags associadas também não são validadas contra o tenant. <pre>val idTenant: Long, val idAccount: Long, ... // UpsertEventDto / UpsertProjectDto createEventAccountRelationship(idEvent, params.idAccount, membershipLabel = "Organizador")</pre> Explorabilidade: Nenhuma condição além de C-01.

3. IDOR

Severidade	Arquivo:linha	Descrição
CRÍTICA	backend/.../synergia/rest/EntityAccountResource.kt:55-63 backend/.../synergia/libs/entityAccount/services/EntityAccountSqlService.kt:86-99 backend/.../synergia/libs/entityAccount/services/EntityAccountService.kt:38-42	C-03 · IDOR em update de conta: troca a senha do ADMIN de qualquer instituição Carrega a conta pelo ID do path e sobrescreve login, e-mail, nome e senha sem verificar quem pede nem se a conta pertence ao tenant. O idTenant do corpo é ignorado. Resultado: tomada de conta total de qualquer usuário, inclusive o ADMIN criado com cada instituição. <pre>accountRepository.findById(idAccount).ifPresent { account -> account.login = params.login ... if (!params.password.isNullOrBlank()) { account.password = params.password } }</pre> Explorabilidade: IDs sequenciais (IDENTITY); os IDs vêm prontos de A-01.
ALTA	backend/.../synergia/rest/EntityDeleteByIdResource.kt:15-22 backend/.../synergia/rest/EntityDeleteByIdResource.kt:23-30	A-05 · Exclusão de contas e tags por ID sem checagem (tag via GET) Apaga qualquer conta ou tag da base. A exclusão de tag é um GET, que pode ser disparado até por uma tag ou por pré-carregamento de links. <pre>@GetMapping("delete-tag-by-id/{id-tag}") -> tagRepository.deleteById(idTag) @DeleteMapping("/delete-account/{id-account}") -> accountRepository.deleteById(idAccount)</pre> Explorabilidade: Nenhuma condição; varredura de IDs apaga a base inteira.
ALTA	backend/.../synergia/rest/EntityEventResource.kt:51-59 backend/.../synergia/libs/entityEvent/services/EntityEventSqlService.kt:102-109 backend/.../synergia/rest/EntityProjectResource.kt:51-59 backend/.../synergia/libs/entityProject/services/EntityProjectSqlService.kt:91-97 backend/.../synergia/rest/EntityTagResource.kt:65-73 backend/.../synergia/libs/entityTag/services/EntityTagSqlService.kt:61-69	A-06 · Update de evento, projeto e tag sem verificação de posse Nenhuma checagem de membro/organizador nem de tenant. As permissões EDIT_*_AS_NON_MEMBER existem no enum mas não são aplicadas. Permite pichação de conteúdo e troca de bannerUrl por imagem do atacante. <pre>eventRepository.findById(idEvent).ifPresent { event -> event.title = params.title ... } val project = projectRepository.findById(idProject).orElseThrow()</pre> Explorabilidade: Nenhuma condição.
ALTA	backend/.../synergia/rest/EntityTenantResource.kt:31-39 backend/.../synergia/libs/entityTenant/services/EntityTenantSqlService.kt:43-49	A-07 · Update de tenant sem checagem: renomeia qualquer instituição Altera título e identifier (chave única exibida no login) de qualquer instituição. Serve para phishing ("FAG" vira "FAG - novo login") ou para bloquear o cadastro de um identifier legítimo. <pre>tenantRepository.findById(idTenant).ifPresent { tenant -> tenant.title = params.title; tenant.identifier = params.identifier }</pre> Explorabilidade: Nenhuma condição.
MÉDIA	backend/.../synergia/rest/EntityGetByIdResource.kt:31-43 backend/.../synergia/rest/EntityGetByIdResource.kt:44-60 backend/.../synergia/rest/EntityGetByIdResource.kt:61-77 backend/.../synergia/rest/EntityGetByIdResource.kt:78-85	M-02 · Leitura por ID de conta, evento, projeto e tag de qualquer tenant findById direto do path. Com lookup-members=true o evento/projeto vem com a lista de membros (login e e-mail). <pre>val el = accountRepository.findById(idAccount).orElse(null)?.toDto()</pre> Explorabilidade: Nenhuma condição.

Severidade	Arquivo:linha	Descrição
MÉDIA	backend/.../synergia/rest/EntityAccountResource.kt:27-35 backend/.../synergia/rest/EntityEventResource.kt:25-33 backend/.../synergia/rest/EntityProjectResource.kt:25-42 backend/.../synergia/rest/EntityTagResource.kt:39-56 backend/.../resources/sql/entity-account/list-accounts-by-event.sql:10 backend/.../resources/sql/entity-account/list-accounts-by-project.sql:10 backend/.../resources/sql/entity-event/list-events-by-account.sql:11 backend/.../resources/sql/entity-project/list-projects-by-account.sql:11 backend/.../resources/sql/page-extended-event/list-tags-of-event.sql:11 backend/.../resources/sql/page-extended-project/list-tags-of-project.sql:11	M-03 · Listagens por relacionamento sem filtro de tenant As queries por evento/projeto/conta filtram só pelo ID do relacionamento. Revela membros, e-mails e participação de qualquer conta em qualquer instituição. <pre>WHERE ear.id_event = :id_event -- nenhum a.id_tenant = ...</pre> Explorabilidade: Nenhuma condição.
INFORMATIVA	backend/.../synergia/rest/AttributePermissionsResource.kt:17-22 backend/.../resources/sql/action-attribute-permissions/attribute-permissions.sql (0 bytes) backend/.../synergia/rest/ActionManagerRelationshipsResource.kt:16-36 backend/.../synergia/libs/actionManagerRelationships/services/ActionManagerRelationshipsSqlService.kt:10-27	I-01 · Endpoints de permissões e relacionamentos sem autorização (hoje inertes) Hoje não são exploráveis: o SQL de atribuição de permissões está vazio (a chamada falha) e os métodos de relacionamento não têm corpo. Assim que forem implementados sem autorização, o primeiro vira escalada de privilégio direta (qualquer um se dá qualquer permissão). <pre>fun attributePermissions(@RequestBody params: AttributePermissionsDto) // sem checagem de papel</pre> Explorabilidade: Latente: depende de os métodos serem implementados.

4. Chaves expostas

Severidade	Arquivo:linha	Descrição
ALTA	frontend/nx.json:20	A-08 · Token do Nx Cloud com escopo read-write versionado em repositório público Presente desde o commit inicial 5b929f2 (10/10/2024). Com escrita no cache remoto, um terceiro pode publicar artefatos de build envenenados que builds futuros reutilizam como cache hit. <pre>"nxCloudAccessToken": "YTLmM2RlNjgt...fHJLYWQtd3JpdGU="</pre> <pre># base64 -> a9f3de68-c3c5-4e66-af91-acc03116ab03 read-write</pre> Explorabilidade: Repositório público no GitHub; o token precisa estar ativo no Nx Cloud.
MÉDIA	backend/.../resources/application-datasource.yaml:4-5 backend/.../resources/application.yaml:8 backend/.../resources/application-e2e.yaml:14-15 backend/build.gradle.kts:89-90 backend/test/.../synergia/integration/support/IntegrationTestBase.kt:44-45 college/dev-ops/atv-01/response-02.md:185 college/dev-ops/atv-01/response-03.md:257,265	M-04 · Credenciais de banco no perfil padrão e como fallback \${VAR:MaybeLater}, sem validação de startup O perfil importado por padrão (application.yaml:8) já traz usuário e senha reais da máquina de desenvolvimento, e os perfis de teste caem no mesmo valor quando a variável falta. Não há checagem na inicialização que recuse esses valores; um jar subido sem as variáveis conecta silenciosamente com eles. A senha está publicada no GitHub e repetida na documentação. <pre>username: raindrop password: MaybeLater # perfil importado por padrão password: \${E2E_DB_PASSWORD:MaybeLater}</pre> Explorabilidade: Postgres acessível com essa senha, ou reuso da senha em outros serviços.

Severidade	Arquivo:linha	Descrição
BAIXA	backend/.../resources/application-datasource-windows.yaml:4-5	B-02 · Credencial padrão postgres/admin no perfil Windows Credencial de superusuário padrão. Baixa porque o perfil está comentado em application.yaml:9, mas incentiva manter o Postgres local com senha trivial. <code>username: postgres password: admin</code> Explorabilidade: Só se o perfil for ativado e o banco estiver exposto.
BAIXA	commit 0b79851 - backend/.../resources/application-evoge.yaml:5 (branch origin/chore/atualiza-dependencias) commits 87702be / 8ce1026 - application-datasource.yaml (password: Admin)	B-03 · Senhas de banco no histórico git O arquivo application-evoge.yaml não está na branch atual, mas continua no remoto público. Apagar o arquivo não remove o segredo; ele precisa ser tratado como vazado. <code>username: evoge_user password: evoge_pass</code> Explorabilidade: Reuso da senha em algum banco acessível.
INFORMATIVA	.github/workflows/ci.yml:33-34 .github/workflows/ci.yml:102-103 .github/workflows/ci.yml:130-132 .github/workflows/ci.yml:169-171	I-02 · Credenciais do Postgres efêmero do CI em texto claro Aceitável: o banco nasce e morre dentro do job e não é acessível de fora. Registrado só para cobertura; não gera issue. <code>POSTGRES_USER: synergia POSTGRES_PASSWORD: synergia</code> Explorabilidade: Não explorável.

5. XSS

Severidade	Arquivo:linha	Descrição
BAIXA	backend/.../synergia/libs/entityEvent/services/EntityEventSqlService.kt:75,106 backend/.../synergia/libs/entityProject/services/EntityProjectSqlService.kt:72,95 frontend/src/libs/components/src/lib/safe-image/safe-image.component.html:4	B-04 · bannerUrl aceita qualquer URL externa (sem XSS, mas rastreia quem visualiza) O Angular sanitiza [src] (javascript: vira unsafe:) e não executa script, então não há XSS. Mas qualquer URL http(s) é aceita e salva: quem abre o card faz uma requisição ao servidor do atacante (IP, User-Agent, horário). Combinado com A-06, dá para trocar o banner de eventos alheios. <code></code> Explorabilidade: Requer criar ou editar um evento/projeto (trivial por C-01).

Achados adicionais (fora das cinco categorias)

Severidade	Arquivo:linha	Descrição
ALTA	backend/.../synergia/libs/UtilsEntities/jpa/account/Account.kt:30-31 backend/.../resources/sql/page-login/check-login-information.sql:18 backend/.../synergia/libs/entityAccount/services/EntityAccountSqlService.kt:73,95 backend/.../synergia/libs/entityTenant/services/EntityTenantSqlService.kt:37	A-X1 · Senhas armazenadas e comparadas em texto puro Fora das cinco categorias, mas relevante: não há hash (bcrypt/argon2). Qualquer vazamento do banco (ou do backup) expõe as senhas de todos os usuários, que costumam ser reutilizadas. <code>ELSE a.password = :password END</code> Explorabilidade: Acesso de leitura ao banco.
BAIXA	backend/.../synergia/libs/UtilsCommons/objects/ResponseMessenger.kt:12-14 backend/.../synergia/libs/UtilsCommons/objects/ResponseMessenger.kt:26-32	B-X1 · Mensagem de exceção devolvida ao cliente no header x-error Erros de JPA/JDBC/Postgres chegam ao cliente com nomes de tabelas, colunas e constraints, o que facilita mapear o esquema. <code>headers.set("x-error", sanitizedMessage) // e.message cru, até 8 KB</code> Explorabilidade: Provocar qualquer erro (ID inexistente, constraint).

Severidade	Arquivo:linha	Descrição
INFORMATIVA	backend/build.gradle.kts:48-49 backend/.../resources/application-s pringdoc.yaml:1-11	I-03 · Swagger UI e /v3/api-docs publicados em produção Com a API sem autenticação, a documentação interativa vira o console do atacante. Não é falha por si, mas deveria ficar desligada fora de desenvolvimento. <pre>implementation("org.springdoc:springdoc-openapi-starter-webmvc-ui:2.5.0")</pre> Explorabilidade: Sempre ativo (nenhuma propriedade springdoc.*.enabled=false).

Recomendações priorizadas

P1 Imediato (dias)

1. Revogar o nxCloudAccessToken no Nx Cloud e passar a injetá-lo por NX_CLOUD_ACCESS_TOKEN (A-08).
2. Remover o ramo checkLastSeen do login; "continuar logado" deve usar um token/refresh emitido pelo servidor (C-02).
3. Enquanto não há autenticação, tirar do ar ou proteger (Basic Auth no proxy do Render) as rotas de update/delete (C-03, A-05, A-07).
4. Trocar a senha MaybeLater em todo lugar onde ela for usada e tratar evoge_pass e Admin como vazadas (M-04, B-03).

P2 Curto prazo (sprint)

1. Adicionar spring-boot-starter-security com sessão ou JWT assinado; o login passa a emitir o token e nenhuma rota fica anônima além de login, cadastro de tenant e listagem pública (C-01).
2. Derivar idTenant e idAccount do principal autenticado em todos os controllers e remover esses campos dos DTOs (A-01, A-02, A-04).
3. Verificar posse/tenant antes de todo findById/save/deleteById, de preferência num helper único que faça findByIdAndIdTenant (A-06, M-02, M-03).
4. Aplicar AuthPermissionsEnum no servidor (@PreAuthorize ou checagem explícita) e só então exibir o menu Administração por papel (A-03, I-01).
5. Armazenar senhas com BCryptPasswordEncoder e migrar as existentes (A-X1).

P3 Médio prazo

1. Filtrar is_private em list-all-tenants e devolver só o necessário para o login (M-01).
2. Validar bannerUrl (só https, domínios permitidos) ou migrar para upload próprio (B-04).
3. Trocar a exclusão de tag para DELETE e responder erros com mensagem genérica + ID de correlação no log (A-05, B-X1).
4. Desligar springdoc fora de desenvolvimento e adicionar validação de startup que recuse credenciais padrão (I-03, M-04).
5. Considerar RLS no Postgres (SET app.tenant_id por requisição) como segunda barreira ao filtro da aplicação.

A ordem P2 importa: a autenticação (item 1) é pré-requisito para que os itens 2 a 4 tenham de onde tirar o tenant e o usuário. Até lá, as medidas de P1 reduzem a exposição.

ISSUES PARA O GITHUB

12 issues prontas para copiar e colar. Cada bloco vai de **--- ISSUE n ---** a **--- FIM ISSUE n ---**: a primeira linha (# ...) é o título; a linha Labels lista as labels sugeridas; o restante é o corpo em Markdown. Achados triviais do mesmo tema foram agrupados; I-02 (credenciais do Postgres efêmero do CI) não virou issue por não ser acionável. Linhas longas foram quebradas para caber na página; o Markdown renderiza igual.

CRÍTICA**Issue 1 · [Segurança] Backend sem autenticação: todos os endpoints aceitam chamadas anônimas**

```
--- ISSUE 1 ---
# [Segurança] Backend sem autenticação: todos os endpoints aceitam chamadas anônimas

Labels: security, severity:critical

## Descrição
O backend não tem Spring Security, filtro, interceptor nem token. Nenhum dos 35 handlers sabe quem está chamando. O `id-tenant` e o `idAccount` usados para filtrar e para atribuir autoria vêm do próprio cliente, e o único controle de acesso é o guard `HasActiveTenant` do Angular, que lê o `localStorage`.

É explorável porque qualquer cliente HTTP (curl, script, Swagger) chama `https://repo-tests.onrender.com/api/...` diretamente; CORS só restringe navegadores.

Achados relacionados: C-01, A-03, A-04 do relatório de auditoria.

## Evidência
- `backend/build.gradle.kts:39-52`: sem `spring-boot-starter-security`.
- `backend/src/main/kotlin/br/com/synergia/rest/EntityAccountResource.kt:16-18`:
  ```kotlin
 @PostMapping("/list-accounts-by-tenant")
 fun listAccountsByTenant(@RequestParam("id-tenant") idTenant: Long, ...)
  ```
- `frontend/src/app/security/routing/has-active-tenant.ts:18-21`:
  ```ts
 if (this.sessionService.getTenantId()) { return true; }
  ```
- `backend/src/main/kotlin/br/com/synergia/libs/entityEvent/models/UpsertEventDto.kt:4-5`: `idTenant` e `idAccount` vêm do corpo.
- `backend/src/main/kotlin/br/com/synergia/libs/utils/commons/enums/AuthPermissionsEnum.kt:8-20`: o enum de permissões nunca é referenciado.

## Impacto
Leitura, criação, alteração e exclusão de dados de todas as instituições por qualquer pessoa na internet. É a causa raiz das issues de isolamento de tenant e IDOR.

## Sugestão de correção
1. Adicionar `spring-boot-starter-security` e emitir um token (JWT assinado com segredo vindo de variável de ambiente, ou sessão com cookie `HttpOnly`/`Secure`/`SameSite`) no login.
2. Configurar um `SecurityFilterChain` que exija autenticação em `/api/**`, liberando só login, cadastro de tenant e o mínimo de listagem pública.
3. Expor o usuário autenticado (`idAccount`, `idTenant`, permissões) como principal e removê-los dos DTOs e parâmetros.
4. Aplicar `AuthPermissionsEnum` com `@PreAuthorize` nas rotas administrativas.
5. No frontend, anexar o token num `HttpInterceptor` e esconder o menu Administração por permissão (apenas UX; o servidor decide).

## Critérios de aceite
- [ ] Requisição sem token a qualquer rota de `/api/**`, exceto as liberadas explicitamente, responde 401.
- [ ] Nenhum controller recebe `id-tenant`/`idTenant`/`idAccount` do cliente para decidir escopo ou autoria.
- [ ] Rotas administrativas respondem 403 para usuário sem a permissão correspondente.
- [ ] Teste de integração cobre 401 (sem token), 403 (sem permissão) e 200 (com permissão).
- [ ] O segredo de assinatura vem de variável de ambiente e a aplicação não sobe sem ele.
--- FIM ISSUE 1 ---
```

CRÍTICA**Issue 2 · [Segurança] Login dispensa a senha quando o cliente envia checkLastSeen=true**

```
--- ISSUE 2 ---
# [Segurança] Login dispensa a senha quando o cliente envia checkLastSeen=true

Labels: security, severity:critical

## Descrição
O endpoint `POST /api/page-login/check-login-information` aceita o booleano `checkLastSeen` do cliente. Quando ele é `true`, o SQL não compara a senha: basta que o usuário tenha logado nas últimas 12 horas.

É explorável porque `idTenant` (via `list-all-tenants`) e `login` (via `list-accounts-by-tenant`) são públicos. Cada login bem-sucedido renova o `last_seen`, então o atacante mantém o acesso indefinidamente.

## Evidência
`backend/src/main/resources/sql/page-login/check-login-information.sql:13-18`
```sql
CASE
 WHEN :check_last_seen THEN (
 a.last_seen is NOT NULL AND
 ((now() - a.last_seen) < interval '12 hours')
)
 ELSE a.password = :password END
...
`frontend/src/app/modules/page-login/route-login.component.ts:53-58`: o frontend envia `password: ''` e `checkLastSeen: true`.

`backend/src/test/kotlin/br/com/synergia/integration/PageLoginIntegrationTest.kt:121-133`: teste que garante o comportamento ("mesmo sem senha").

Impacto
Entrar como qualquer usuário ativo, inclusive o `ADMIN` de cada instituição, sem conhecer a senha.

Sugestão de correção
- Remover `checkLastSeen` do DTO e o ramo `CASE` do SQL; o login sempre exige senha.
- Implementar "continuar logado" com um refresh token opaco emitido pelo servidor, guardado em cookie `HttpOnly`, com expiração e revogação.
- Inverter o teste de integração para exigir que o login sem senha falhe.

Critérios de aceite
- [] `LoginInformationInputDto` não tem mais `checkLastSeen`.
- [] Login com senha errada ou vazia retorna falha independentemente de `last_seen`.
- [] O teste "sessao recente é aceita ... mesmo sem senha" foi substituído por um teste que espera recusa.
- [] Reabrir o app após login recente funciona via token emitido pelo servidor, não via `localStorage` com IDs.
--- FIM ISSUE 2 ---
```

**CRÍTICA****Issue 3 · [Segurança] IDOR em update de conta permite trocar a senha de qualquer usuário**

```
--- ISSUE 3 ---
[Segurança] IDOR em update de conta permite trocar a senha de qualquer usuário

Labels: security, severity:critical

Descrição
`POST /api/entity-account/update/{id-account}` carrega a conta pelo ID do path e sobrescreve login, e-mail, nome e senha sem verificar quem pede nem se a conta é do mesmo tenant. O `idTenant` do corpo é ignorado.

É explorável porque os IDs são sequenciais e a lista de contas de qualquer tenant é pública.

Evidência
`backend/src/main/kotlin/br/com/synergia/libs/entityAccount/services/EntityAccountSqlService.kt:86-99`
```kotlin
accountRepository.findById(idAccount).ifPresent { account ->
    account.email = params.email
    account.login = params.login
    if (!params.password.isNullOrBlank()) {
        account.password = params.password
    }
    accountRepository.save(account)
}
...

```

```
`backend/src/main/kotlin/br/com/synergia/rest/EntityAccountResource.kt:55-63`
```

Impacto

Tomada de conta total, inclusive do `ADMIN` de cada instituição; o dono legítimo perde o acesso.

Sugestão de correção

- Permitir a edição só para a própria conta ou para quem tem `CREATE\_USER` no mesmo tenant.
- Buscar com `findByIdAndIdTenant(id, principal.idTenant)` e responder 404 quando não encontrar.
- Troca de senha da própria conta deve exigir a senha atual.

Critérios de aceite

- [] Usuário A não consegue alterar a conta de B (403/404), no mesmo ou em outro tenant.
 - [] Administrador de um tenant não altera contas de outro tenant.
 - [] Trocar a própria senha exige a senha atual.
 - [] Testes de integração cobrem os três casos.
- FIM ISSUE 3 ---

ALTA

Issue 4 · [Segurança] Listagens por tenant confiam no id-tenant enviado pelo cliente

--- ISSUE 4 ---

[Segurança] Listagens por tenant confiam no id-tenant enviado pelo cliente

Labels: security, severity:high

Descrição

As listagens filtram por `id\_tenant`, mas o valor vem do parâmetro `id-tenant` da requisição. Trocar o número dá acesso aos dados de outra instituição. `list-all-tenants` entrega a lista de IDs, inclusive de instituições privadas, e `get-account-by-login-or-email` confirma a existência de logins/e-mails.

Achados: A-01, A-02, M-01, B-01.

Evidência

```
- `backend/src/main/kotlin/br/com/synergia/rest/EntityAccountResource.kt:16-26` e
  `backend/src/main/resources/sql/page-list-accounts/list-accounts.sql:9`
```sql
WHERE a.id_tenant = :id_tenant
```
- `backend/src/main/kotlin/br/com/synergia/rest/EntityEventResource.kt:15-24`
- `backend/src/main/kotlin/br/com/synergia/rest/EntityProjectResource.kt:15-24`
- `backend/src/main/kotlin/br/com/synergia/rest/EntityTagResource.kt:21-38`
- `backend/src/main/kotlin/br/com/synergia/libs/entityTenant/services/EntityTenantSqlService.kt:19-21`
```kotlin
return tenantRepository.findAll().map { it.toDto() }
```
- `backend/src/main/kotlin/br/com/synergia/rest/EntityAccountResource.kt:36-46`
```

Impacto

Vazamento de login, nome e e-mail de todos os usuários e do conteúdo de todas as instituições, inclusive as privadas. Os logins alimentam o bypass de senha.

Sugestão de correção

- Depende da issue de autenticação: usar `principal.idTenant` em vez do parâmetro e remover `id-tenant` das assinaturas.
- `list-all-tenants`: devolver só `id`/`title` de instituições não privadas, ou exigir o identifier exato.
- `get-account-by-login-or-email`: restringir a usuários autenticados do mesmo tenant.

Critérios de aceite

- [] Nenhum endpoint de listagem aceita `id-tenant` do cliente.
 - [] Usuário do tenant A recebe lista vazia/403 ao tentar dados do tenant B (teste de integração).
 - [] `list-all-tenants` não retorna tenants com `is\_private = true`.
 - [] `get-account-by-login-or-email` responde 401 sem autenticação.
- FIM ISSUE 4 ---

ALTA

Issue 5 · [Segurança] Update e delete por ID sem verificação de posse (eventos, projetos, tags, contas, tenants)

--- ISSUE 5 ---

[Segurança] Update e delete por ID sem verificação de posse (eventos, projetos, tags, contas, tenants)

Labels: security, severity:high

Descrição

Os handlers de escrita carregam o objeto pelo ID do path e alteram/apagam sem verificar tenant, membro ou papel. A exclusão de tag é um `GET`.

Achados: A-05, A-06, A-07.

```
## Evidência
- `backend/src/main/kotlin/br/com/synergia/rest/EntityDeleteByIdResource.kt:15-30`
```kotlin
@GetMapping("delete-tag-by-id/{id-tag}") -> tagRepository.deleteById(idTag)
@DeleteMapping("/delete-account/{id-account}") -> accountRepository.deleteById(idAccount)
```

- `backend/src/main/kotlin/br/com/synergia/libs/entityEvent/services/EntityEventSqlService.kt:102-109`
- `backend/src/main/kotlin/br/com/synergia/libs/entityProject/services/EntityProjectSqlService.kt:91-97`
- `backend/src/main/kotlin/br/com/synergia/libs/entityTag/services/EntityTagSqlService.kt:61-69`
- `backend/src/main/kotlin/br/com/synergia/libs/entityTenant/services/EntityTenantSqlService.kt:43-49`
```kotlin
tenantRepository.findById(idTenant).ifPresent { tenant ->
 tenant.title = params.title
 tenant.identifier = params.identifier
}

Impacto
Qualquer pessoa apaga contas e tags, altera eventos/projetos/tags de outras instituições e renomeia instituições (phishing no seletor de login).

Sugestão de correção
- Criar um helper de autorização (`findOwnedOrThrow`) que busca por `id` + `idTenant` do principal e confere membro/organizador ou a permissão `EDIT_*_AS_NON_MEMBER` / `DELETE_*_AS_NON_OWNER`.
- Update de tenant só para o `ADMIN` daquele tenant.
- Trocar `GET delete-tag-by-id` por `DELETE`.
- Validar que as tags associadas (`params.tags`) pertencem ao mesmo tenant.

Critérios de aceite
- [] Update/delete de objeto de outro tenant responde 404.
- [] Update de evento/projeto por não membro sem permissão responde 403.
- [] Update de tenant só é aceito para o ADMIN do próprio tenant.
- [] Não existe mais rota `GET` que apague dados.
- [] Testes de integração cobrem cada rota de escrita com usuário de outro tenant.
--- FIM ISSUE 5 ---
```

**MÉDIA****Issue 6 · [Segurança] Leitura por ID e por relacionamento sem filtro de tenant**

```
--- ISSUE 6 ---
[Segurança] Leitura por ID e por relacionamento sem filtro de tenant

Labels: security, severity:medium

Descrição
`get-*-by-id` e as listagens por evento/projeto/conta filtram só pelo ID do objeto ou do relacionamento. Com `lookup-members=true`, eventos e projetos vêm com login e e-mail dos membros.

Achados: M-02, M-03.

Evidência
- `backend/src/main/kotlin/br/com/synergia/rest/EntityGetByIdResource.kt:31-85`
```kotlin
val el = accountRepository.findById(idAccount).orElse(null)?.toDto()
```

- `backend/src/main/resources/sql/entity-account/list-accounts-by-event.sql:10`
```sql
WHERE ear.id_event = :id_event
```

- `list-accounts-by-project.sql:10`, `list-events-by-account.sql:11`, `list-projects-by-account.sql:11`, `list-tags-of-event.sql:11`, `list-tags-of-project.sql:11`.

Impacto
Enumeração de membros, e-mails e participação de qualquer conta em qualquer instituição.

Sugestão de correção
- Adicionar `AND <tabela>.id_tenant = :id_tenant` com o tenant do principal em todas as queries por relacionamento.
- Trocar `findById` por `findByIdAndIdTenant` em `EntityGetByIdResource`.

Critérios de aceite
- [] Toda query em `backend/src/main/resources/sql` que retorna dados de tenant tem filtro por `id_tenant` do principal.
- [] `get-*-by-id` de objeto de outro tenant responde 404.
- [] Teste de integração cobre leitura cruzada entre dois tenants.
--- FIM ISSUE 6 ---
```



ALTA

**Issue 7 · [Segurança] Token do Nx Cloud com escopo read-write versionado em repositório público**

```
--- ISSUE 7 ---
[Segurança] Token do Nx Cloud com escopo read-write versionado em repositório público

Labels: security, severity:high

Descrição
`frontend/nx.json` contém um `nxCloudAccessToken` que decodifica para
`a9f3de68-c3c5-4e66-af91-acc03116ab03|read-write`. Ele está no repositório desde o commit inicial
`5b929f2` (10/10/2024) e o repositório é público.

Evidência
`frontend/nx.json:20`
```json
"nxCloudAccessToken": "YtlmM2RlNjgtYzNjNS00ZTY2LWVmOTEtYWNjMDMxMTZhYjAzfHJlYWQtd3JpdGU="
```

Impacto
Com escrita no cache remoto, um terceiro pode publicar artefatos de build adulterados que builds futuros aceitam como cache hit (envenenamento de cache / cadeia de suprimentos). Também consome a cota da conta.

Sugestão de correção
1. Revogar o token no painel do Nx Cloud.
2. Remover a chave do `nx.json` e usar a variável `NX_CLOUD_ACCESS_TOKEN` (secret no CI, `.env` local fora do git).
3. Se o Nx Cloud for mantido, usar token read-only para desenvolvedores e read-write só no CI.
4. Adicionar varredura de segredos (gitleaks ou GitHub secret scanning / push protection).

Critérios de aceite
- [] O token antigo está revogado e uma chamada com ele falha.
- [] `frontend/nx.json` não contém `nxCloudAccessToken`.
- [] O CI obtém o token de `secrets.*`.
- [] Varredura de segredos roda no CI ou via push protection.
--- FIM ISSUE 7 ---
```

ALTA

**Issue 8 · [Segurança] Senhas de usuários armazenadas e comparadas em texto puro**

```
--- ISSUE 8 ---
[Segurança] Senhas de usuários armazenadas e comparadas em texto puro

Labels: security, severity:high

Descrição
A coluna `account.password` guarda a senha como veio do cliente, e o login compara com `=` no SQL. Não há bcrypt/argon2.

Evidência
- `backend/src/main/kotlin/br/com/synergia/libs/utillsEntities/jpa/account/Account.kt:30-31`
- `backend/src/main/resources/sql/page-login/check-login-information.sql:18`
```sql
ELSE a.password = :password END
```
- `backend/src/main/kotlin/br/com/synergia/libs/entityAccount/services/EntityAccountSqlService.kt:73,95`
- `backend/src/main/kotlin/br/com/synergia/libs/entityTenant/services/EntityTenantSqlService.kt:37`

Impacto
Qualquer leitura do banco (backup, dump, SQL injection futura, acesso indevido ao Render) expõe as senhas reais de todos os usuários.

Sugestão de correção
- Usar `BCryptPasswordEncoder` (ou Argon2) do Spring Security para gravar e `matches()` para verificar, no código e não no SQL.
- Migração: na próxima autenticação bem-sucedida, regravar como hash; forçar troca de senha para contas não migradas após um prazo.

Critérios de aceite
- [] Nenhuma senha em texto puro no banco (consulta de verificação retorna 0 linhas sem prefixo `$2`).
- [] O SQL de login não compara senha.
- [] Teste de integração confirma que o valor gravado difere da senha enviada e que o login funciona.
--- FIM ISSUE 8 ---
```

## MÉDIA

**Issue 9 · [Segurança] Credenciais de banco padrão no código, na documentação e no histórico git**

```
--- ISSUE 9 ---
[Segurança] Credenciais de banco padrão no código, na documentação e no histórico git

Labels: security, severity:medium

Descrição
O perfil importado por padrão traz usuário/senha reais da máquina de desenvolvimento, e os perfis de teste usam o mesmo valor como fallback quando a variável não existe. Não há validação de startup que recuse esses valores. Outras senhas estão no histórico git.

Achados: M-04, B-02, B-03.

Evidência
- `backend/src/main/resources/application-datasource.yaml:4-5` (importado por `application.yaml:8`)
  ```yaml
  username: raindrop
  password: MaybeLater
  ```
- `backend/src/main/resources/application-e2e.yaml:14-15`: `${E2E_DB_PASSWORD:MaybeLater}`
- `backend/build.gradle.kts:89-90` e
 `backend/src/test/kotlin/br/com/synergia/integration/support/IntegrationTestBase.kt:44-45`: fallback `MaybeLater`
- `college/dev-ops/atv-01/response-02.md:185`, `college/dev-ops/atv-01/response-03.md:257,265`
- `backend/src/main/resources/application-datasource-windows.yaml:4-5`: `postgres` / `admin`
- Histórico: commit `0b79851` (`application-evoge.yaml:5`, `evoge_pass`, branch `origin/chore/atualiza-dependencias`); commits `87702be` / `8ce1026` (`password: Admin`).

Impacto
Uma instância subida sem as variáveis conecta com credenciais conhecidas publicamente. Senhas publicadas tendem a ser reutilizadas em outros serviços.

Sugestão de correção
1. Trocar as senhas `MaybeLater`, `evoge_pass` e `Admin` onde estiverem em uso.
2. Remover valores reais dos YAML: `${DB_PASSWORD}` sem default, com `.env.example` documentando as variáveis.
3. Adicionar validação na inicialização (ex.: `@PostConstruct` ou `EnvironmentPostProcessor`) que aborta fora do perfil de teste se a senha estiver vazia ou numa lista de valores padrão.
4. Opcional: reescrever o histórico (git filter-repo) só depois de rotacionar; a rotação é o que protege.

Critérios de aceite
- [] Nenhum arquivo versionado contém `MaybeLater`, `evoge_pass` ou `password: admin`.
- [] A aplicação não sobe sem `DB_PASSWORD` definido (fora do perfil de teste).
- [] As senhas antigas foram trocadas nos bancos que as usavam.
- [] Documentação usa placeholders (`<sua-senha>`).
--- FIM ISSUE 9 ---
```

## BAIXA

**Issue 10 · [Segurança] bannerUrl aceita qualquer URL externa e expõe visitantes a rastreamento**

```
--- ISSUE 10 ---
[Segurança] bannerUrl aceita qualquer URL externa e expõe visitantes a rastreamento

Labels: security, severity:low

Descrição
`bannerUrl` de eventos e projetos é gravado sem validação e renderizado em `
```

```
(`@Pattern` + checagem de host).
- Alternativa: upload para storage próprio e servir pela aplicação.
- Adicionar `referrerpolicy="no-referrer"` na `<img>` e uma CSP com `img-src` restrito.

Critérios de aceite
- [] Backend rejeita `bannerUrl` que não seja `https://` de domínio permitido (400).
- [] Teste unitário cobre `http:`, `javascript:`, `data:` e domínio não permitido.
- [] Frontend exibe a cor de fallback para URLs rejeitadas.
--- FIM ISSUE 10 ---
```

**BAIXA****Issue 11 · [Segurança] Mensagens de exceção e Swagger expostos publicamente**

```
--- ISSUE 11 ---
[Segurança] Mensagens de exceção e Swagger expostos publicamente

Labels: security, severity:low

Descrição
Toda exceção vira o header `x-error` com a mensagem crua (até 8 KB), incluindo erros de JPA/Postgres com nomes de tabelas e constraints. O Swagger UI e `/v3/api-docs` estão ativos em produção.

Achados: B-X1, I-03.

Evidência
- `backend/src/main/kotlin/br/com/synergia/libs/UtilsCommons/objects/ResponseMessenger.kt:12-14,26-32`
 ``kotlin
 errorTemplate(truncateForHeader((e.message ?: "Erro desconhecido.")))
 headers.set("x-error", sanitizedMessage)
 ``
- `backend/build.gradle.kts:48-49`: `springdoc-openapi-starter-webmvc-ui`, sem
 `springdoc.*.enabled=false` em nenhum perfil.

Impacto
Facilita o reconhecimento do esquema e da superfície da API.

Sugestão de correção
- Mensagens de negócio: exceção própria (`BusinessException`) cuja mensagem pode ir ao cliente; demais
 erros: texto genérico + ID de correlação, com o detalhe só no log.
- Trocar `Content-Type: text/html` das respostas de erro por `application/problem+json`.
- No perfil de produção: `springdoc.api-docs.enabled=false` e `springdoc.swagger-ui.enabled=false`.

Critérios de aceite
- [] Erro de banco provocado em produção devolve mensagem genérica sem nomes de tabela/constraint.
- [] `GET /swagger-ui.html` e `/v3/api-docs` respondem 404 no perfil de produção.
- [] O log contém o ID de correlação e a exceção completa.
--- FIM ISSUE 11 ---
```

**INFORMATIVA****Issue 12 · [Segurança] Exigir autorização nos endpoints de permissões e relacionamentos antes de implementá-los**

```
--- ISSUE 12 ---
[Segurança] Exigir autorização nos endpoints de permissões e relacionamentos antes de implementá-los

Labels: security, severity:info

Descrição
`POST /api/action-attribute-permissions/attribute-permissions` e os três endpoints de
`action-manage-relationships` não verificam papel nem tenant. Hoje são inertes: o arquivo SQL de
atribuição está vazio e os métodos de relacionamento não têm corpo. Quando forem implementados como
estão, o primeiro permite que qualquer um conceda a si mesmo qualquer permissão.

Evidência
- `backend/src/main/kotlin/br/com/synergia/rest/ActionAttributePermissionsResource.kt:17-22`
- `backend/src/main/resources/sql/action-attribute-permissions/attribute-permissions.sql` (0 bytes)
- `backend/src/main/kotlin/br/com/synergia/rest/ActionManageRelationshipsResource.kt:16-36`
- `backend/src/main/kotlin/br/com/synergia/libs/actionManageRelationships/services/ActionManageRelations
hipsServiceImpl.kt:10-27`

Impacto
Latente: escalada de privilégio e vínculo de objetos entre tenants quando a funcionalidade for
concluída.

Sugestão de correção
- Implementar junto com a checagem: só quem tem permissão administrativa no tenant atribui permissões;
 `idAccounts` e `idPermissions` validados contra o tenant do principal.
- Relacionamentos: exigir que evento, projeto e tag sejam do tenant do principal e que o chamador seja
 membro/organizador.
```

```
Critérios de aceite
- [] Usuário sem permissão administrativa recebe 403 em `attribute-permissions`.
- [] Não é possível atribuir permissões a contas de outro tenant.
- [] Relacionar objetos de tenants diferentes responde 400/404.
- [] Testes de integração cobrem os três casos antes do merge da funcionalidade.
--- FIM ISSUE 12 ---
```